onstruct Binary Tree from Preorder and Inorder Tr

Trees | LeetCode: https://leetcode.com/problems/construct-binary-tree-from-pr

Problem

- Given preorder and inorder traversals of a tree with unique values, rebuild the tree.

Intuition

- Preorder[0] is the root. Its position in inorder splits the remaining values into the left subtree (before it) and the right subtree (after it).
- The left subtree size tells us how many of the following preorder values belong to the left.

Brute Force

- Search the root in inorder linearly and slice arrays each time: O(n^2) time and memory.

Optimal Approach

- Map value → inorder index.
- Build(inLo, inHi) uses a global preorder pointer: root = pre[p++]; mid = index[root].
- Root.left = build(inLo, mid-1); root.right = build(mid+1, inHi). Left must be built first because preorder lists left before right.

Worked Example

```
pre = [3,9,20,15,7], in = [9,3,15,20,7].

root 3 (in idx 1): left in[0..0]  right in[2..4]
  left: root 9 leaf
  right: root 20 (in idx 3): left 15, right 7
tree [3,9,20,null,null,15,7]
```

Java Solution

```java
public class ConstructBinaryTreeFromPreorderInorder {
    private int p;
    private int[] pre;
    private Map<Integer, Integer> idx;

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        pre = preorder;
        p = 0;
        idx = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) idx.put(inorder[i], i);
        return build(0, inorder.length - 1);
    }

    private TreeNode build(int lo, int hi) {
        if (lo > hi) return null;
        TreeNode root = new TreeNode(pre[p++]);
        int mid = idx.get(root.val);
        root.left = build(lo, mid - 1);
        root.right = build(mid + 1, hi);
        return root;
    }
}
```

Complexity

- Time: O(n). Space: O(n) for the map plus O(h) recursion.

Pitfalls

- Building right before left consumes preorder in the wrong order.